

Android-Labor-01

Android Development with Kotlin

Objectives

- Kotlin basics
- Variables (val, var), string templates, ranges
- Functions
- Compact functions, lambdas
- Arrays, lists and their transformations: `listOf`, `mutableListOf`, `arrayOf`, `Array`, `intArrayOf`, `IntArray`, `filter`, `map`, `forEach`, `any`, `all`, `sortedBy`

Exercises

1. String template

A **template expression** starts with a dollar sign `$` and can be a simple value `$value` or an expression inside curly braces `${expression}`.

Write a `main` function that adds two immutable variables and prints the result using a string template in the following format: `2 + 3 = 5`.

2. Filtering lists

- Write a main function that declares an immutable list (`listOf`) **daysOfWeek** containing the days of the week.
- Use a `for` loop that iterates over the list and prints the list to the standard output.
- Use a list `filter` to print the days starting with letter 'T'
- Use a list `filter` to print the days containing the letter 'e'
- Use a list `filter` to print all the days of length 6 (e.g. Friday)

3. Functions (1)

Write a function `isPrime` that checks whether a number is prime. Then, write a `main` function that prints all prime numbers within a given range.

4. Functions (2)

Write an **encode** and a corresponding **decode** function that encode and decode the characters of a string, respectively. You may use any encoding strategy.

- Test your functions!
- Write a **higher-order function** (takes a function as parameter) that encodes or decodes a message. Call this function twice. Once for encoding and once for decoding a message.

```
fun messageCoding(msg: String, func: (String) -> String): String
```

5. Compact Functions

Write a **compact function** (single-expression function) that prints the even numbers from a list. Use a list **filter**!

Compact function example:

```
fun double(x: Int): Int = x * 2
```

6. Transformations: **map**

The **map** performs the same transformation on every list item and returns the result list.

Using **map**, perform the following operations:

- Double the elements of a list of integers and print it.
- Print the days of week capitalized (e.g. MONDAY for Monday)
- Print the first character of each day capitalized (e.g. m for Monday)
- Print the length of days (number of characters, e.g. Monday → 6)
- Compute the average length of days (in number of characters)

7. Mutable lists.

- Convert the immutable list **daysOfWeek** into a mutable list.
- Remove all days that contain the letter **n**, then print the resulting list. The output should be:

```
[Tuesday, Thursday, Friday, Saturday]
```

- Print each element of the list in a new line together with the index of the element (convert the list to list with index using the **withIndex()** function!). You should get the following result:

```
Item at 0 is Tuesday
Item at 1 is Thursday
Item at 2 is Friday
Item at 3 is Saturday
```

- Sort the result list alphabetically! You should get the following result:

```
[Friday, Saturday, Thursday, Tuesday]
```

8. Arrays

- Generate an array of 10 random integers between 0 and 100, and use `forEach` to print each element of the array in a new line.
- Print the array sorted in ascending order!
- Check whether the array contains any even number!
- Check whether all the numbers are even!
- Calculate the average of generated numbers and print it using `forEach`!

9. Anagrams

Source

Given an array of strings, group the anagrams together. You can return the answer in any order.

An anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Use the following test class to verify your solution!

```
class AnagramsGrouperTest {

    @Test
    fun threeGroupsAllLowerCase() {
        val anagrams = groupAnagrams(listOf("eat", "tea", "tan", "ate",
"nat", "bat").toTypedArray())
        assertEquals(3, anagrams.size)
        assertTrue(anagrams.contains(listOf("eat", "tea", "ate")))
        assertTrue(anagrams.contains(listOf("tan", "nat")))
        assertTrue(anagrams.contains(listOf("bat")))
    }

    @Test
    fun threeGroupsSomeUpperCase() {
        val anagrams = groupAnagrams(listOf("eat", "tEa", "Tan", "atE",
"NAT", "bat").toTypedArray())
        assertEquals(3, anagrams.size)
        assertTrue(anagrams.contains(listOf("eat", "tea", "ate")))
        assertTrue(anagrams.contains(listOf("tan", "nat")))
        assertTrue(anagrams.contains(listOf("bat")))
    }

    @Test
    fun validOneGroup() {
        val anagrams = groupAnagrams(listOf("eat").toTypedArray())
        assertEquals(1, anagrams.size)
        assertTrue(anagrams.contains(listOf("eat")))
    }

    @Test
```

```
fun noGroup() {  
    val anagrams = groupAnagrams(emptyList<String>().toTypedArray())  
    assertEquals(0, anagrams.size)  
}  
}
```